



Figure 5: Action statements

As an example, consider the file, 'data':

```

12  abcd 12.2
13  mnop 11.1
•   awk '{printf("@%d %-3s %0.3f", $1, $2, $3)}' data
    the above command appends an @ before first field, left
    assign second field, print third field with 3 decimal places
    o/p
@12abcd 12.200
@13mnop 11.100
  
```

**Decision statements:** An *if-else* decision statement evaluates an expression and takes proper action. Nested *if* statements are also applied.

As an example, to print all records with more than three fields, type:

```

BEGIN{}
{
  If(NF > 3)
    print $0
  else
    print "Less than 3 fields"
}
  
```

To print the marks and grades of a student, type:

```

BEGIN(print "Mark & grade of a student")
{
  If($2=="Amy")
    S=S+$3
}
END(print "Total marks: "S
if(S>50)
  print "Grade A"
else
  print "Grade B"
}
  
```

**Loop statements:** *While*, *do.. while* and *for* are the loop statements in AWK. The AWK *while* loop checks

the condition first, and if the condition is true, it executes the list of actions. This process repeats until the condition becomes false.

Here is an example:

```

BEGIN {print "Display even numbers from 10 to 20" }
{ #initialization
  I = 10
  #loop limit test
  while (I <=20)
  { #action
    print I
    I+=2      #update
  }
} # end script
do.. while loop
  
```

The AWK *do while* loop executes the body once, then repeats the body as long as the condition is true. Here is an example that displays numbers from 1 to 5:

```

awk 'BEGIN {I=1; do {print i; i++ } while(i < 5)} '
  
```

Here is an example of the *for* loop:

```

program name : awkfor
BEGIN { print " Sum of fields in all lines"}
{
  for ( i=1; i<=NF; i++)
  {
    t=t+$i    //sum of $1 + sum of $2
  }
}
END { print "Sum is " t}
Consider the input file : data
10 30
10 20
Running the script : awk -f awkfor data
Sum of fields in all lines
Sum is 70
  
```

**Control statements:** *next*, *getline* and *exit* are the control statements in AWK. The '*next*' statement alters the flow of the program — it stops the current processing of pattern space. The program reads the next line and starts executing commands with the new line.

*Getline* is similar to *next*, but continues executing the script.

The *exit* statement causes AWK to immediately stop processing the input, and any remaining lines are ignored.

## Mathematical functions in AWK

The various mathematical functions in AWK are:

*int(x)* -- truncates the floating point to the integer

*cos(x)* -- returns the cosine of x

*exp(x)* -- returns  $e^x$